L12_DAAN862

November 15, 2025

Lesson 12 Assignment

DAAN 862: Data Analytics

Instructor: Prof. Guang Yang

Eric Smith

Penn State University

---

[6]:
```
# ------------------------------------------------------------------------
# Question 1 - Load Text
# ------------------------------------------------------------------------
BaseFilePath = Path(r"D:\GradSchool\DAAN862\Lesson12\Assignment_12.txt")

file = open(BaseFilePath , 'r')

text1 = file.read()

file.close()

print("\n=== Output 500 characters to verifty import ===")
print(text1[:500])
```

```
=== Output 500 characters to verifty import ===
arxiv_annotate10_7_1.txt   arxiv_annotate10_7_2.txt   arxiv_annotate10_7_3.txt
arxiv_annotate1_13_1.txt   arxiv_annotate1_13_2.txt   arxiv_annotate1_13_3.txt
```

```
arxiv_annotate2_66_1.txt   arxiv_annotate2_66_2.txt   arxiv_annotate2_66_3.txt
arxiv_annotate3_80_1.txt   arxiv_annotate3_80_2.txt   arxiv_annotate3_80_3.txt
arxiv_annotate4_168_1.txt   arxiv_annotate4_168_2.txt
arxiv_annotate4_168_3.txt   arxiv_annotate5_240_1.txt
arxiv_annotate5_240_2.txt   arxiv_annotate5_240_3.txt   arxiv_an
```

Question 1 - Load data

Data was successfully loaded as output shows.

[7]:
```
# ------------------------------------------------------------------------
# Question 2 - File-name Pattern & Match Count
# ------------------------------------------------------------------------

import re


pattern = r'[A-Z0-9._%+-]+\.[A-Z]{2,4}'

# Compile with IGNORECASE, just like the Lesson 12 email example
regex = re.compile(pattern, flags=re.IGNORECASE)

# Find all filenames that match this pattern anywhere in text1
filenames_q2 = regex.findall(text1)

print("\n=== File-name Pattern & Match Count ===")
print("Pattern used for filenames:", regex.pattern)
print("Number of filenames matching the pattern:", len(filenames_q2))
print("First 10 matches:", filenames_q2[:10])
```

```
=== File-name Pattern & Match Count ===
Pattern used for filenames: [A-Z0-9._%+-]+\.[A-Z]{2,4}
Number of filenames matching the pattern: 90
First 10 matches: ['arxiv_annotate10_7_1.txt', 'arxiv_annotate10_7_2.txt',
'arxiv_annotate10_7_3.txt', 'arxiv_annotate1_13_1.txt',
'arxiv_annotate1_13_2.txt', 'arxiv_annotate1_13_3.txt',
'arxiv_annotate2_66_1.txt', 'arxiv_annotate2_66_2.txt',
'arxiv_annotate2_66_3.txt', 'arxiv_annotate3_80_1.txt']
```

Question 2 – File-name Pattern & Match Count

Using regular expressions from Lesson 12, the pattern `[A-Z0-9._%+-]+[A-Z]{2,4}` was designed to capture typical file names ending with a 2–4-letter extension. When applied to the text, 90 file names matched this pattern. This confirms that most entries in the document follow a consistent naming rule such as `arxiv_annotate10_7_1.txt`.

[8]:
```
# ------------------------------------------------------------------------
# Question 3 - Filenames That Do NOT Match the Pattern
# ------------------------------------------------------------------------

# Split the text into candidate "filenames" (likely one per line or separated␣
 ↪by spaces)
all_names = text1.split()   # whitespace split is enough for this assignment

non_matching = []

for name in all_names:
    # fullmatch: the whole token must match the pattern to count as a "valid"␣
 ↪filename
    if regex.fullmatch(name) is None:
        non_matching.append(name)

print("\n=== Filenames That Do NOT Match the Pattern ===")
print("Total tokens (candidate filenames):", len(all_names))
print("Number of filenames that DO NOT match the pattern:", len(non_matching))
print("First 10 non-matching names:", non_matching[:10])
```

```
=== Filenames That Do NOT Match the Pattern ===
Total tokens (candidate filenames): 90
Number of filenames that DO NOT match the pattern: 5
First 10 non-matching names: ['jdm_annˆotate3_120_1.txt',
'jdm_anno&tate6_32_2.txt', 'jdm_annotat#e8_177_2.txt',
'plos_annotat*e1_6_2.txt', 'plos_annot@ate7_1233_2.txt']
```

Question 3 – Filenames That Do Not Match the Pattern

All 90 tokens were checked against the same pattern using `fullmatch`. Only five file names did not fit the expected structure because they contained unusual characters like `^`, `&`, or `@`. These exceptions highlight minor formatting errors in the dataset but confirm that nearly all file names follow the intended standard.

[16]:
```
# ------------------------------------------------------------------------
# Question 4 - Word Normalization & Counts
# ------------------------------------------------------------------------
import nltk

BaseFilePath2 = Path(r"D:\GradSchool\DAAN862\Lesson12\arxiv_annotate1_13_1.txt")
file = open(BaseFilePath2, 'r')

text2 = file.read()

file.close()
```

```
tokens = nltk.word_tokenize(text2)

tokens_lower = [w.lower() for w in tokens]

dist = nltk.FreqDist(tokens_lower)

print("\n=== Word Normalization & Counts ===")
print("Total number of normalized tokens:", len(tokens_lower))
print("Number of unique normalized words:", len(dist))
print("Most common normalized words and their counts:")
print(dist.most_common(20))
```

```
=== Word Normalization & Counts ===
Total number of normalized tokens: 856
Number of unique normalized words: 333
Most common normalized words and their counts:
[('the', 44), ('of', 34), ('as', 28), ('and', 24), ('misc', 20), ('we', 20),
('a', 19), ('in', 19), ('to', 18), ('number', 16), ('internet', 15), ('#', 12),
('different', 12), ('that', 12), ('ases', 12), ('ownx', 12), ('types', 10),
('with', 9), ('on', 9), ('topology', 8)]
```

Question 4 – Word Normalization & Counts

The text was tokenized and converted to lowercase to normalize word forms. Using `nltk.FreqDist`, a total of 856 normalized tokens were identified, representing 333 unique words. Common words such as *the*, *of*, and *as* appeared most frequently, showing that the file contains typical English text with repeated connective terms.